CS3070 – Exploration 5 -- Inter-Process Communication
Summer 2025

SCORE: 26.0/30

Questions

1. Many OSs attempted to provide IPC using shared memory. That is, they tried to avoid a message-exchange system in favor of a limited exception to the rule that address spaces cannot overlap. The exception was that a single page of real memory could be mapped into a page of the address spaces of each process involved in the sharing. Despite its apparent simplicity, this turns out to have been a bad idea. We ask you to explore why in the questions below. Review the kernel calls needed to implement this form of shared memory in ArtOS2 Module 5.3. The goal is that with these kernel operations, the processes in a group can now read and write the shared page as if it were in its address space.

   a. Do all the processes in the group need to use the same page number for the shared page?

   3.0 No, all the processes in the group do not have to use the same page number. Each process assigns a page within their own page table to point to the shared page (frame). Between the processes page and the shared page, there exists an intermediary descriptor to resolve the issue of modifying the shared page and having to then modify each individual process page entry.

   b. How does the operating system know when it is time to delete the shared page?

   3.0 The OS maintains a reference count for each shared page that tracks how many processes have it mapped. When a process calls EXIT_GROUP(i,g) or terminates, the reference count decrements. The shared page is deleted when the reference count reaches zero, indicating no processes are using it. The creator can also call DISSOLVE_GROUP(g) to force deletion after all members exit.

   c. What must happen if the OS decides to page out the shared page, then reload it later in a different real memory page frame?

   2.0 The OS must update all page table entries across all processes that reference the old physical frame to point to the new frame location. This requires maintaining reverse mappings from physical pages to virtual pages - complex bookkeeping that violates address space isolation. The descriptor indirection solution works: update only the shared descriptor instead of individual process page tables, but this adds translation overhead and demonstrates why shared memory complicates the clean virtual memory model.

   d. How do processes using the shared page avoid race conditions?

Processes must implement their own synchronization since the OS provides no automatic coordination for shared memory access. Each process uses semaphores around shared page operations:

3.0

WAIT(S)

use page

SIGNAL(S)

This pushes complex concurrency control to user level where it's error-prone and violates the principle of keeping synchronization in the protected kernel space.

e. Suppose a process sets out to copy array N bytes to the shared page, beginning at offset 0 of the shared pages. What happens if N is larger than the page size? What problems does this cause?

1.5

This creates an overflow problem, as the process that writes to the shared page now is writing in "unknown territory" since there are no restrictions. The overflow bytes that are either written are read from exist in the next address space, which is not shared memory. Each of the processes have no idea what exists in that overflow memory, which makes this problem catastrophic if we are overwriting critical data.

2. A private semaphore is a semaphore owned by a specific process that only the owning process can wait on. [Review Ch 3.3 of Module 3 near the end.] Private semaphores are useful for the protocols between user processes and system service processes. One way to implement private semaphore is to reserve the first *K* semaphores as the private semaphores in a system that allows a maximum of *K* processes. In other words, **SEM[i]** is the private semaphore of process **i** and only process **i** is allowed to issue **WAIT(i)**. However, any other process can issue **SIGNAL(i)**.

a. Many OS provide kernel calls **SLEEP** (to put the caller into the sleep state) and **WAKEUP(k)** to awaken process **k** from its sleep. Express **SLEEP** and **WAKEUP** as operations on private semaphores.

3.0

SLEEP = WAIT(psem[self]) where psem[self] is the calling process's private semaphore

WAKEUP(p) = SIGNAL(psem[p]) where psem[p] is process p's private semaphore

b. Familiarize yourself with the pseudo code for a more efficient version of **SLEEP** and **WAKEUP** with private semaphores. The *counter* and *queue* are replaced with two bits, *waiting*, and *wakeup-waiting*. Why are these two bits sufficient to represent the counter & queue states from part (b)? Where are these two bits stored?

3.0

- Sufficiency: The two bits capture all necessary states for process synchronization while avoiding the lost wakeup problem:

    - waiting bit: Indicates the process is blocked/sleeping (process moved from READY to BLOCKED state)
    - wakeup-waiting bit: Prevents race condition where WAKEUP occurs before SLEEP
- Three possible states:
    - waiting=0, wakeup-waiting=0: Process is READY, no pending wakeup signal (same as counter = 1 and empty queue)
    - waiting=1, wakeup-waiting=0: Process is BLOCKED (sleeping), waiting for signal (same as counter <1 and empty queue)
    - waiting=0, wakeup-waiting=1: Pending wakeup exists, next SLEEP will not block (same counter <1 and caller in queue)
- The fourth state (waiting=1, wakeup-waiting=1) is impossible by design - a process cannot simultaneously be blocked and have a pending wakeup.
- The bits are stored in the Process Control Block (PCB) or Thread Control Block (TCB).

3. The virtual memory completely isolates processes by guaranteeing that their address spaces can never overlap. This isolation is fundamental to the proofs that processes cannot interfere with one another through hidden memory channels. However, there are many situations when processes must communicate. Level 4 for IPC (Inter-process Communication) does this by providing a message-exchange system that is outside all address spaces. A process can send a message to another process by inserting it in a buffer provided by the IPC level, which places the buffer in the inbox of the receiving process. In ArtOS2, review Module 5.2, which is a common use case for message exchange. It defines a protocol between a user process, a disk controller process, and a disk.

    a. As a team, walk through this code, discussing it carefully until every team member understands the purpose and operation of each semaphore and the data transfers. Attest here you have done this.

    3.0

    I attest that as a team; we discussed the code and understood the purpose of the operation of each semaphore and the data transfers.

    b. Is it possible for these processes to deadlock while waiting for each other to respond to requests? A deadlock is a circular wait among processes that freezes all of them because none can give a signal to release any other. If not, prove it. If so, give an example of a deadlock.

    No, it is not possible for these processes to deadlock while waiting for each other to respond to requests.

    3.0

    Assume: p, s, and the device could be in a deadlock.

    Proof:

    - In a deadlock, circular wait occurs among processes that freezes all of them because none can give a signal to release any other

- p waits on s, s waits on device, device waits on p
- In the provided code, p waits for s(GET_REPLY)
- s waits for the device SLEEP after start(driver,mb.message)
- The device never waits and it only signals completion (DCH -> WAKEUP(s))
- The device cannot be a part of a circular wait since the chain is strictly p -> s -> device.

Contradiction: Circular wait is impossible; therefore, deadlock is not possible among these processes.

c. Is it possible that multiple processes could deadlock in the SEND operation where they stop and wait for a message buffer from the system pool? How can such a deadlock be prevented?

Yes. A deadlock is possible in the SEND operation.

1.5

- Each SEND requires an empty message buffer from the system pool
- If the pool size is K, but the request is for K + 1 space, SEND will find no buffer available.
- Earlier processes would be asleep in GET_REPLY holding their buffer and waiting for a reply.
- The reply will never come because the last SEND didn't succeed.
- This creates the circular wait condition because every process is waiting on the next in the chain, but no one can release a buffer.
- Multiple processes can deadlock the SEND operation

Reserve dedicated buffers for replies so they cannot be consumed by outgoing requests, ensuring the reply path remains unblocked. Additionally, implement timeouts on SEND operations to break circular waits, or use priority-based buffer allocation where high-priority processes can preempt buffers from lower-priority processes. The key is preventing all buffers from being held by processes waiting for replies that can never arrive.